Fundamentals of Deep Learning and Programming

Lecture 6: Convolutional Neural Network (CNN) (2) and Introduction to Natural Language Processing (NLP)

Fall Semester 2025

Adjunct Professor: Donghoon Kang

Conolutional Neural Network (CNN) (2)

Matrix

(1) Addition

$$\begin{bmatrix} 2 & 1 & 0 \\ 3 & 4 & 1 \end{bmatrix} + \begin{bmatrix} -1 & 1 & 3 \\ 0 & 0 & 7 \end{bmatrix} = \begin{bmatrix} 1 & 2 & 3 \\ 3 & 4 & 8 \end{bmatrix}.$$

$$[1 \quad 2] + [0 \quad -1] = [1 \quad 1].$$

(2) Scalar multiplication

$$3 \begin{bmatrix} 1 & -1 & 2 \\ 0 & 1 & 5 \\ 1 & 0 & 3 \end{bmatrix} = \begin{bmatrix} 3 & -3 & 6 \\ 0 & 3 & 15 \\ 3 & 0 & 9 \end{bmatrix}.$$

$A = [a_{ij}]$

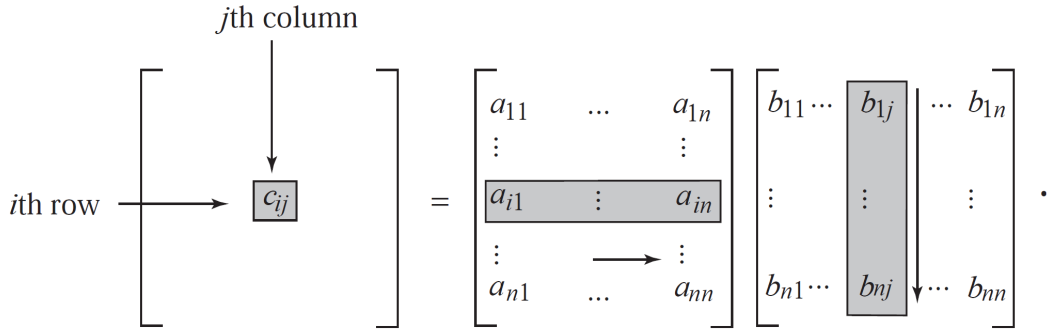
$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}.$$

$m \times n$ matrix

Next we turn to matrix multiplication. If $A = [a_{ij}]$, $B = [b_{ij}]$ are $n \times n$ matrices, then the product $AB = C$ has entries given by

$$c_{ij} = \sum_{k=1}^{n} a_{ik} b_{kj},$$

which is the dot product of the ith row of A and the jth column of B:



Similarly, we can multiply an $m \times n$ matrix (m rows, n columns) by an $n \times p$ matrix (n rows, p columns) to obtain an $m \times p$ matrix (m rows, p columns) by the same rule.

Example 1

$$A = \begin{bmatrix} 1 & 0 & 3 \\ 2 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix} \quad \text{and} \quad B = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 1 \end{bmatrix}.$$

Then

$$AB = \begin{bmatrix} 0 & 4 & 3 \\ 1 & 2 & 0 \\ 0 & 1 & 0 \end{bmatrix} \quad \text{and} \quad BA = \begin{bmatrix} 2 & 1 & 0 \\ 1 & 0 & 3 \\ 3 & 1 & 0 \end{bmatrix}.$$

Observe that $AB \neq BA$. ▲

Example 2

$$A = \begin{bmatrix} 2 & 0 & 1 \\ 1 & 1 & 2 \end{bmatrix} \quad \text{and} \quad B = \begin{bmatrix} 1 & 0 & 2 \\ 0 & 2 & 1 \\ 1 & 1 & 1 \end{bmatrix}.$$

$$AB = \begin{bmatrix} 3 & 1 & 5 \\ 3 & 4 & 5 \end{bmatrix}, \quad \text{and } BA \text{ is not defined.}$$

1 Column Convention in Mathematics

Traditional linear algebra often uses the **column convention**, where each input sample is represented as a column vector. In this case, the linear transformation is written as:

$$y = Wx + b \tag{1}$$

Here:

x	Input matrix of shape (D, B), where D is the number of input features and B is the batch size
W	Weight matrix of shape (O, D), where O is the number of output features
b	Bias vector of shape $(O, 1)$, broadcasted across all B samples
y	Output matrix of shape (O, B)

Thus, $x \in \mathbb{R}^{D \times B}$, $W \in \mathbb{R}^{O \times D}$, and $y \in \mathbb{R}^{O \times B}$.

2 Row Convention in PyTorch

In contrast, PyTorch follows the **row convention**. Each row in a tensor represents one data sample, and each column represents a feature. For an input tensor $x \in \mathbb{R}^{B \times D}$, where B is the batch size and D is the number of input features, a linear layer is defined as:

$$y = xW^T + b \tag{2}$$

Here:

x	Input tensor of shape (B, D)
W	Weight matrix of shape (O, D)
b	Bias vector of shape (O)
y	Output tensor of shape (B, O)

Thus, $x \in \mathbb{R}^{B \times D}$, $W \in \mathbb{R}^{O \times D}$, and $y \in \mathbb{R}^{B \times O}$.

Framework	Convention	Formula	Input Shape	Weight Shape	Output Shape
PyTorch / TensorFlow / NumPy	Row	$y = xW^T + b$	(B, D)	(O, D)	(B, O)
Math / MATLAB	Column	$y = Wx + b$	(D, B)	(O, D)	(O, B)

Table 3: Comparison of Row vs Column Conventions

Therefore, PyTorch uses the **row convention** by default. Each sample is a row vector, and the linear transformation in PyTorch is implemented as $y = xW^T + b$.

Simple CNN (1)

Example 1

- Input tensor: $[B, 3, 32, 32]$ (batch of RGB images)
- Output tensor: $[B, 10]$ (class logits)

```python
import torch
import torch.nn as nn
import torch.nn.functional as F

class SimpleCNN(nn.Module):
    def __init__(self):
        super().__init__()  # Run the initialization code of nn.Module before adding my own layers.

        # 1st convolutional layer: input channels=3 (RGB), output channels=16
        self.conv1 = nn.Conv2d(in_channels=3, out_channels=16, kernel_size=3, padding=1)
        # 2nd convolutional layer: input=16, output=32
        self.conv2 = nn.Conv2d(16, 32, kernel_size=3, padding=1)

        # Max pooling layer
        self.pool = nn.MaxPool2d(2, 2)

        # Fully connected layers
        self.fc1 = nn.Linear(32 * 8 * 8, 128)  # CIFAR10 images are 32x32
        self.fc2 = nn.Linear(128, 10)  # 10 classes

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))  # Conv1 → ReLU → Pool
        x = self.pool(F.relu(self.conv2(x)))  # Conv2 → ReLU → Pool
        x = x.view(-1, 32 * 8 * 8)            # Flatten
        x = F.relu(self.fc1(x))
        x = self.fc2(x)
        return x

model = SimpleCNN()
print(model)
```

Stage	Output Shape
Input	$[B, 3, 32, 32]$
Conv1 + ReLU + Pool	$[B, 16, 16, 16]$
Conv2 + ReLU + Pool	$[B, 32, 8, 8]$
Flatten	$[B, 2048]$
FC1 + ReLU	$[B, 128]$
FC2	$[B, 10]$

simple_cnn_1.py

```
SimpleCNN(
  (conv1): Conv2d(3, 16, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (conv2): Conv2d(16, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (pool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (fc1): Linear(in_features=2048, out_features=128, bias=True)
  (fc2): Linear(in_features=128, out_features=10, bias=True)
)
```

- `torch`: Core PyTorch library.
- `torch.nn`: Provides building blocks for neural networks such as `Linear` and `Conv2d`.
- `torch.nn.functional`: Functional API that provides stateless versions of operations such as `relu()` and `max_pool2d()`.

1. **Conv1 + ReLU + Pool:** Input $[B, 3, 32, 32] \rightarrow$ Output $[B, 16, 16, 16]$.
2. **Conv2 + ReLU + Pool:** Input $[B, 16, 16, 16] \rightarrow$ Output $[B, 32, 8, 8]$.
3. **Flatten:** Reshape into a 1D vector of size $32 \times 8 \times 8 = 2048$ per sample.
4. **FC1 + ReLU:** Linear transformation to 128 hidden units.
5. **FC2:** Final output of size 10 (class logits).

Cross-Entropy Loss

1 Overview

We derive the gradients of the cross-entropy loss for both binary (sigmoid) and multiclass (softmax) classification in full detail. We also contrast them with mean squared error (MSE) to clarify why cross-entropy is preferred for classification.

Notation. For an input $x \in \mathbb{R}^d$ and parameters (W, b), define the *logit(s)* z as an affine function of x:

$$z = xW + b. \tag{1}$$

In the binary case, $z \in \mathbb{R}$. In the multiclass case with K classes, $z \in \mathbb{R}^K$ and z_k denotes the logit for class k. Targets are denoted by y (either scalar $y \in \{0, 1\}$ or a one-hot / probability vector $y \in \Delta^{K-1}$).

2 Binary Classification (Sigmoid + Binary Cross-Entropy)

2.1 Definitions

Let

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}, \tag{2}$$

and the binary cross-entropy (log loss)

$$L(\hat{y}, y) = -\Big[y \log \hat{y} + (1 - y) \log(1 - \hat{y})\Big]. \tag{3}$$

2.2 Derivative of Sigmoid

Recall the classic identity

$$\frac{d\,\sigma(z)}{dz} = \sigma(z)\big(1 - \sigma(z)\big) = \hat{y}(1 - \hat{y}). \tag{4}$$

2.3 Step-by-step: $\partial L/\partial z$

We use the chain rule:

$$\frac{\partial L}{\partial z} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z}. \tag{5}$$

From (3),

$$\begin{aligned} \frac{\partial L}{\partial \hat{y}} &= -\left[y \cdot \frac{1}{\hat{y}} + (1 - y) \cdot \frac{1}{1 - \hat{y}} \cdot (-1)\right] \\ &= -\frac{y}{\hat{y}} + \frac{1 - y}{1 - \hat{y}}. \end{aligned} \tag{6}$$

Using (4),

$$\frac{\partial \hat{y}}{\partial z} = \hat{y}(1-\hat{y}). \tag{7}$$

Therefore,

$$\begin{aligned}\frac{\partial L}{\partial z} &= \left(-\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}}\right)\hat{y}(1-\hat{y}) \\ &= -y(1-\hat{y}) + (1-y)\hat{y} \\ &= \hat{y} - y. \end{aligned} \tag{8}$$

Key result: For binary logistic regression with cross-entropy, the gradient w.r.t. the logit z is simply $\hat{y} - y$.

2.4 Gradients w.r.t. Parameters

If $z = w^\top x + b$ (scalar z), then by the chain rule

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial z}\frac{\partial z}{\partial w} = (\hat{y} - y)\, x, \tag{9}$$

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial z}\frac{\partial z}{\partial b} = \hat{y} - y. \tag{10}$$

3 Multiclass Classification (Softmax + Cross-Entropy)

3.1 Definitions

For $z \in \mathbb{R}^K$, the softmax is

$$\hat{y}_k = \text{softmax}(z)_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}} \quad (k = 1, \ldots, K). \tag{11}$$

The cross-entropy loss is

$$L(\hat{y}, y) = -\sum_{i=1}^{K} y_i \log \hat{y}_i. \tag{12}$$

Here y can be one-hot (hard label) or any distribution over classes (soft label); in both cases $\sum_i y_i = 1$.

3.2 Derivative of Softmax

Let $S = \sum_{j=1}^{K} e^{z_j}$. Then for any j, k,

$$\begin{aligned} \frac{\partial \hat{y}_j}{\partial z_k} &= \frac{\partial}{\partial z_k}\left(\frac{e^{z_j}}{S}\right) = \frac{\delta_{jk} e^{z_j}\, S - e^{z_j} e^{z_k}}{S^2} \\ &= \frac{e^{z_j}}{S}\left(\delta_{jk} - \frac{e^{z_k}}{S}\right) = \hat{y}_j \left(\delta_{jk} - \hat{y}_k\right), \end{aligned} \tag{13}$$

where δ_{jk} is the Kronecker delta.

3.3 Step-by-step: $\partial L/\partial z_k$

By the chain rule,

$$\frac{\partial L}{\partial z_k} = \sum_{j=1}^{K} \frac{\partial L}{\partial \hat{y}_j} \cdot \frac{\partial \hat{y}_j}{\partial z_k}. \tag{14}$$

From (12),

$$\frac{\partial L}{\partial \hat{y}_j} = -\frac{y_j}{\hat{y}_j}. \tag{15}$$

Combine (15) and (13):

$$\begin{aligned}
\frac{\partial L}{\partial z_k} &= \sum_{j=1}^{K} \left(-\frac{y_j}{\hat{y}_j}\right) \hat{y}_j \left(\delta_{jk} - \hat{y}_k\right) \\
&= -\sum_{j=1}^{K} y_j \delta_{jk} + \sum_{j=1}^{K} y_j \hat{y}_k \\
&= -y_k + \hat{y}_k \sum_{j=1}^{K} y_j \\
&= \hat{y}_k - y_k, \qquad (16)
\end{aligned}$$

since $\sum_j y_j = 1$.

Vector form. Writing $\hat{\boldsymbol{y}}, \boldsymbol{y} \in \mathbb{R}^K$, and $\boldsymbol{z} \in \mathbb{R}^K$, we have

$$\nabla_{\boldsymbol{z}} L = \hat{\boldsymbol{y}} - \boldsymbol{y}. \tag{17}$$

If $\boldsymbol{z} = Wx + b$ with $W \in \mathbb{R}^{K \times d}$ and $b \in \mathbb{R}^K$, then

$$\frac{\partial L}{\partial W} = (\hat{\boldsymbol{y}} - \boldsymbol{y})\, x^\top, \tag{18}$$

$$\frac{\partial L}{\partial b} = \hat{\boldsymbol{y}} - \boldsymbol{y}. \tag{19}$$

3.4 Stable "with logits" form and LogSumExp

In practice we avoid explicitly forming $\hat{\boldsymbol{y}}$ before taking logs. Using the *LogSumExp* (LSE) trick,

$$L = -z_t + \log \sum_{j=1}^{K} e^{z_j}, \tag{20}$$

where t is the index of the true class (for one-hot labels). Differentiating,

$$\frac{\partial L}{\partial z_k} = -\delta_{kt} + \frac{e^{z_k}}{\sum_j e^{z_j}} = \hat{y}_k - \delta_{kt}, \tag{21}$$

which is again $\hat{y}_k - y_k$ when y is one-hot. This is numerically stable and is what many libraries implement (e.g., `softmax_cross_entropy_with_logits`).

4 Why MSE is Problematic for Classification

4.1 Binary case

Take the MSE

$$L_{\text{MSE}} = \frac{1}{2}(\hat{y} - y)^2 \tag{22}$$

with $\hat{y} = \sigma(z)$. Then

$$\frac{\partial L_{\text{MSE}}}{\partial z} = (\hat{y} - y) \cdot \frac{\partial \hat{y}}{\partial z} = (\hat{y} - y)\,\hat{y}(1 - \hat{y}). \tag{23}$$

The additional factor $\hat{y}(1 - \hat{y})$ *saturates* as $\hat{y} \to 0$ or $\hat{y} \to 1$, shrinking the gradient and slowing learning (or causing it to stall). In contrast, cross-entropy yields (8) without this extra shrinking factor.

4.2 Multiclass case

With softmax outputs and MSE,

$$L_{\text{MSE}} = \frac{1}{2} \sum_{k=1}^{K} \left(\hat{y}_k - y_k\right)^2, \tag{24}$$

the gradient w.r.t. logits acquires the softmax Jacobian factor (cf. (13)), again leading to small, entangled gradients near saturation. Cross-entropy cleanly yields $\nabla_{\boldsymbol{z}} L = \hat{\boldsymbol{y}} - \boldsymbol{y}$.

Natural Language Processing (NLP)

Natural Language Processing (NLP)

NLP is the field of computer science that deals with the interaction between computers and human language.

(examples) Think of tasks like spam filters, virtual assistants, or machine translation.

Challenge **Text to Numbers**

Computer can't directly understand raw text like ‘I like cats.’

We need a way to **convert words into numerical data**—Tensors, which PyTorch uses. This is called Text Preprocessing.".

Text Preprocessing

Tokenization Breaking text into smaller units (e.g., words or subwords)

Simple BoW (1)

```python
import torch
import torch.nn as nn

# ----- 1. Vocabulary & Example Sentence -----
vocab = ["I", "like", "cats", "dogs", "apples"]
vocab_size = len(vocab)

# Represent "I like cats." as a BoW vector
bow_vector = torch.tensor([[1., 1., 1., 0., 0.]])  # shape: [1, 5]

# Example label (e.g., Positive sentiment)
label = torch.tensor([[1.]])

# ----- 2. Define a Simple Model -----
class BoWClassifier(nn.Module):
    def __init__(self, input_dim, hidden_dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, 1),
            nn.Sigmoid()
        )
    def forward(self, x):
        return self.net(x)

model = BoWClassifier(input_dim=vocab_size, hidden_dim=4)
```

```python
# ----- 3. Forward Pass -----
output = model(bow_vector)
print("Input (BoW):", bow_vector)
print("Output (Predicted probability):", output.item())
```

```
Input (BoW): tensor([[1., 1., 1., 0., 0.]])
Output (Predicted probability): 0.6920701265335083
```

simple_BoW_1.py

Simple BoW (2)

The code implements a logistic regression model that classifies short text sentences into two categories (positive/negative) using BoW vectors.

```python
import torch
import torch.nn as nn
import torch.nn.functional as F
from typing import List, Dict

######################################################################
# 1. BoW Utility Functions
######################################################################

def build_vocab(sentences: List[str]) -> Dict[str, int]:
    """Builds a word:index vocabulary from the given sentences."""
    words = []
    for sentence in sentences:
        # Simple tokenization: convert to lowercase and split by space
        words.extend(sentence.lower().split())

    # Assign a unique index to each unique word
    word_to_ix = {word: i for i, word in enumerate(set(words))}
    return word_to_ix
```

simple_BoW_2.py

- Converts all sentences to lowercase and splits each into words.
- Builds a dictionary mapping each unique word to an index:

$$\text{word\_to\_ix} = \{\text{word} : i \mid i \in [0, |\text{vocab}|)\}$$

Simple BoW (2)

```
def make_bow_vector(sentence: str, word_to_ix: Dict[str, int]) -> torch.Tensor:
    """Converts a string into a BoW frequency vector."""
    vocab_size = len(word_to_ix)
    # Initialize a zero vector with the size of the vocabulary
    vec = torch.zeros(vocab_size)

    # Simple tokenization
    for word in sentence.lower().split():
        if word in word_to_ix:
            # Increment the count for the word's index (frequency count)
            ix = word_to_ix[word]
            vec[ix] += 1

    # Return as a Tensor, adding a batch dimension (1)
    return vec.view(1, -1)
```

- Initializes a zero vector of size equal to the vocabulary size.
- For each token in the sentence, increments the corresponding position in the vector.
- Returns a frequency-based BoW tensor of shape $(1, V)$, where V is the vocabulary size.

Simple BoW (2)

```
################################################################################
# 2. PyTorch Model Definition
################################################################################

class BoWClassifier(nn.Module):
    """
    A simple text classifier (Logistic Regression) that accepts BoW vectors.
    """
    def __init__(self, vocab_size: int, num_labels: int):
        super(BoWClassifier, self).__init__()
        # Input size = vocabulary size, Output size = number of labels
        self.linear = nn.Linear(vocab_size, num_labels)

    def forward(self, bow_vec: torch.Tensor) -> torch.Tensor:
        # Pass through the linear layer and apply log Softmax
        return F.log_softmax(self.linear(bow_vec), dim=1)
```

- A simple `nn.Module` implementing a linear transformation:

$$y = xW^T + b$$

  where x is the BoW vector and W, b are trainable parameters.

- The model returns $\log(\text{softmax}(xW^T + b))$, suitable for `NLLLoss`.

The combination of `log_softmax` and `NLLLoss` is numerically stable and equivalent to using `CrossEntropyLoss`.

Simple BoW (2)

```
##############################################################################
# 3. Execution and Example
##############################################################################

# Example Dataset (Dog/Cat Theme)
# Label 0: negative, Label 1: positive
data = [("I love my dog, he is friendly", 1),
        ("The cat scratched the furniture", 0),
        ("Dogs are the best companions", 1),
        ("A messy cat made a terrible mess", 0)]

sentences = [item[0] for item in data]
labels = [item[1] for item in data]

# Build Vocabulary
word_to_ix = build_vocab(sentences)
VOCAB_SIZE = len(word_to_ix)
NUM_LABELS = 2  # Two classes: 0 (negative), 1 (positive)

# Instantiate the Model
model = BoWClassifier(VOCAB_SIZE, NUM_LABELS)

# Define Loss Function and Optimizer
loss_function = nn.NLLLoss() # Negative Log Likelihood Loss for log_softmax output
optimizer = torch.optim.SGD(model.parameters(), lr=0.001)
```

- Training data consists of 4 sentences labeled as positive (1) or negative (0).
- Vocabulary is built using `build_vocab()`, and its size determines the input dimension of the model.

Simple BoW (2)

```
# --- Training Loop Example (Mini-training) ---
print(f"Starting Training on {len(data)} samples...")
for epoch in range(10):  # 10 epochs
    for sentence, label in zip(sentences, labels):
        # Step 1. Prepare data (BoW vector) and target (label tensor)
        bow_vec = make_bow_vector(sentence, word_to_ix)
        target = torch.LongTensor([label])
            # torch.LongTensor([label]) converts that
            # integer into a 1D PyTorch tensor containing that value.

        # Step 2. Clear previous gradients
        model.zero_grad()

        # Step 3. Forward pass
        log_probs = model(bow_vec)

        # Step 4. Compute Loss, gradients, and update weights
        loss = loss_function(log_probs, target)
        loss.backward()
        optimizer.step()

    # print(f"Epoch {epoch+1} Loss: {loss.item():.4f}")

print("Training finished.")
```

For each sentence:

1. Convert text into BoW tensor using `make_bow_vector`.
2. Compute log probabilities: $\log P(y|x)$.
3. Compute loss via `NLLLoss`.
4. Backpropagate and update weights.

zip( )

```
sentences = ["I love dogs", "Cats are mean", "Dogs are loyal"]
labels = [1, 0, 1]

pairs = zip(sentences, labels) # creates an iterator that produces these tuples one by one:

print(list(pairs))
```

```
[('I love dogs', 1), ('Cats are mean', 0), ('Dogs are loyal', 1)]
```

Simple BoW (2)

```
# --- Inference Example ---
new_sentence = "My friendly dog loves the companions"
bow_vector_test = make_bow_vector(new_sentence, word_to_ix)

with torch.no_grad(): # Disable gradient calculation for inference
    log_probs_test = model(bow_vector_test)
    predicted_label = torch.argmax(log_probs_test).item()

# Map predicted index to a meaningful label
label_map = {0: "Negative", 1: "Positive"}

print("\n--- Inference Result ---")
print(f"Test Sentence: '{new_sentence}'")
print(f"Log Probabilities: {log_probs_test}")
print(f"Predicted Class: {predicted_label} ({label_map.get(predicted_label)})")
```

```
Starting Training on 4 samples...
Training finished.

--- Inference Result ---
Test Sentence: 'My friendly dog loves the companions'
Log Probabilities: tensor([[-0.6806, -0.7058]])
Predicted Class: 0 (Negative)
```

A new sentence is vectorized using the same vocabulary and passed through the model without gradient computation:

$$\text{log\_probs} = \log(\text{softmax}(xW^T + b)) \tag{1}$$

$$\text{predicted\_label} = \arg\max(\text{log\_probs}) \tag{2}$$

The predicted label is mapped to “Positive” or “Negative” using a dictionary.

Thank you!